OMCS6250
Computer Networks

Intradomain Routing

With Instructor Dr. Maria Konte

Control Plane, Intradomain Routing,
Link-State vs Distance Vector Routing Algorithms

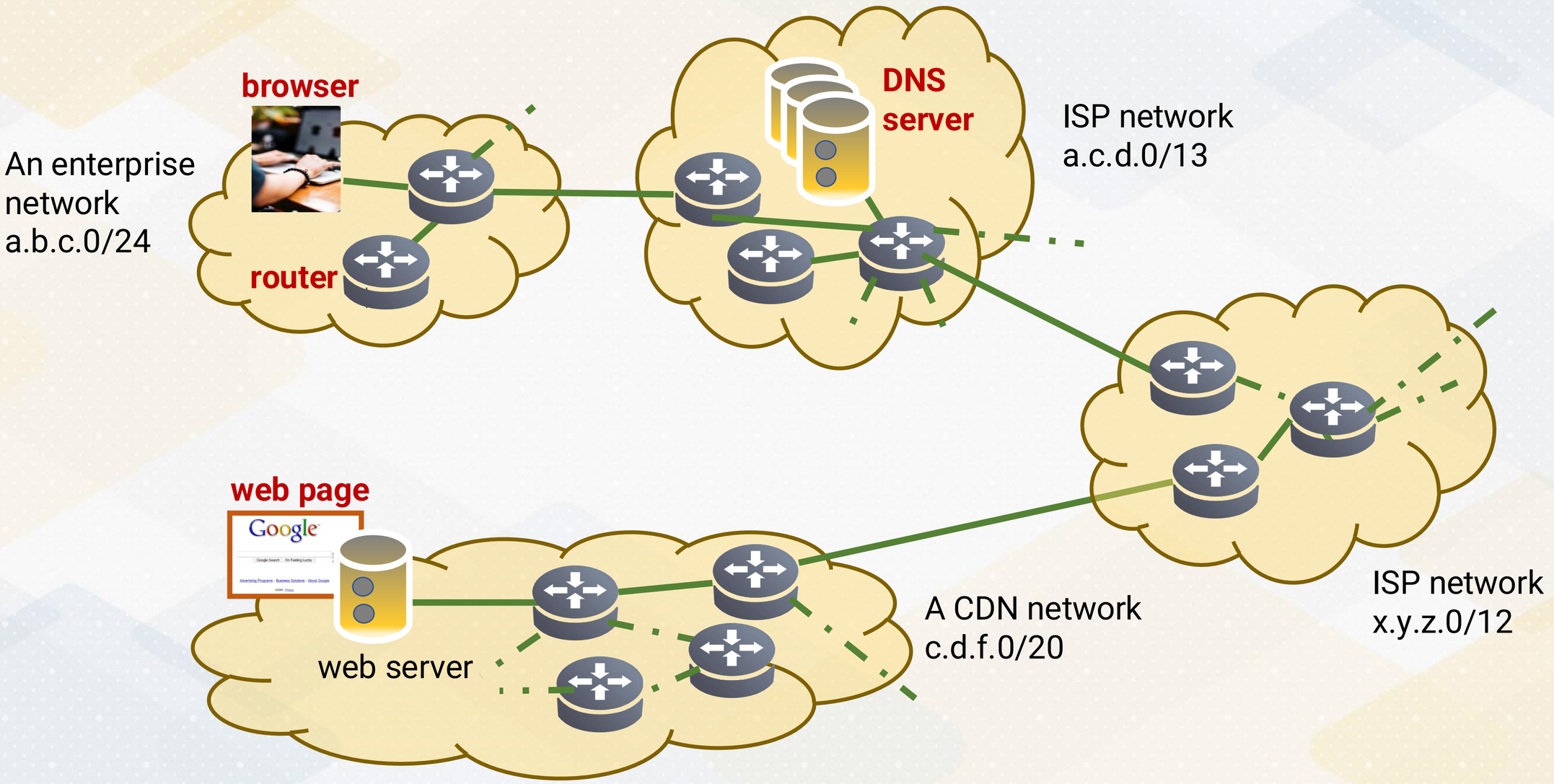
Georgia Tech

Learning Objectives

- **Transport vs. IP**: where routing fits
- What routers do: **routing vs. forwarding**
- Distinguish **intradomain** vs. **interdomain** routing
- Compare **link-state** vs. **distance-vector**
- Walk through how each approach computes routes (Dijkstra vs. Bellman-Ford)
- Recognize real-world challenges: **count-to-infinity, poison reverse, hot potato routing**

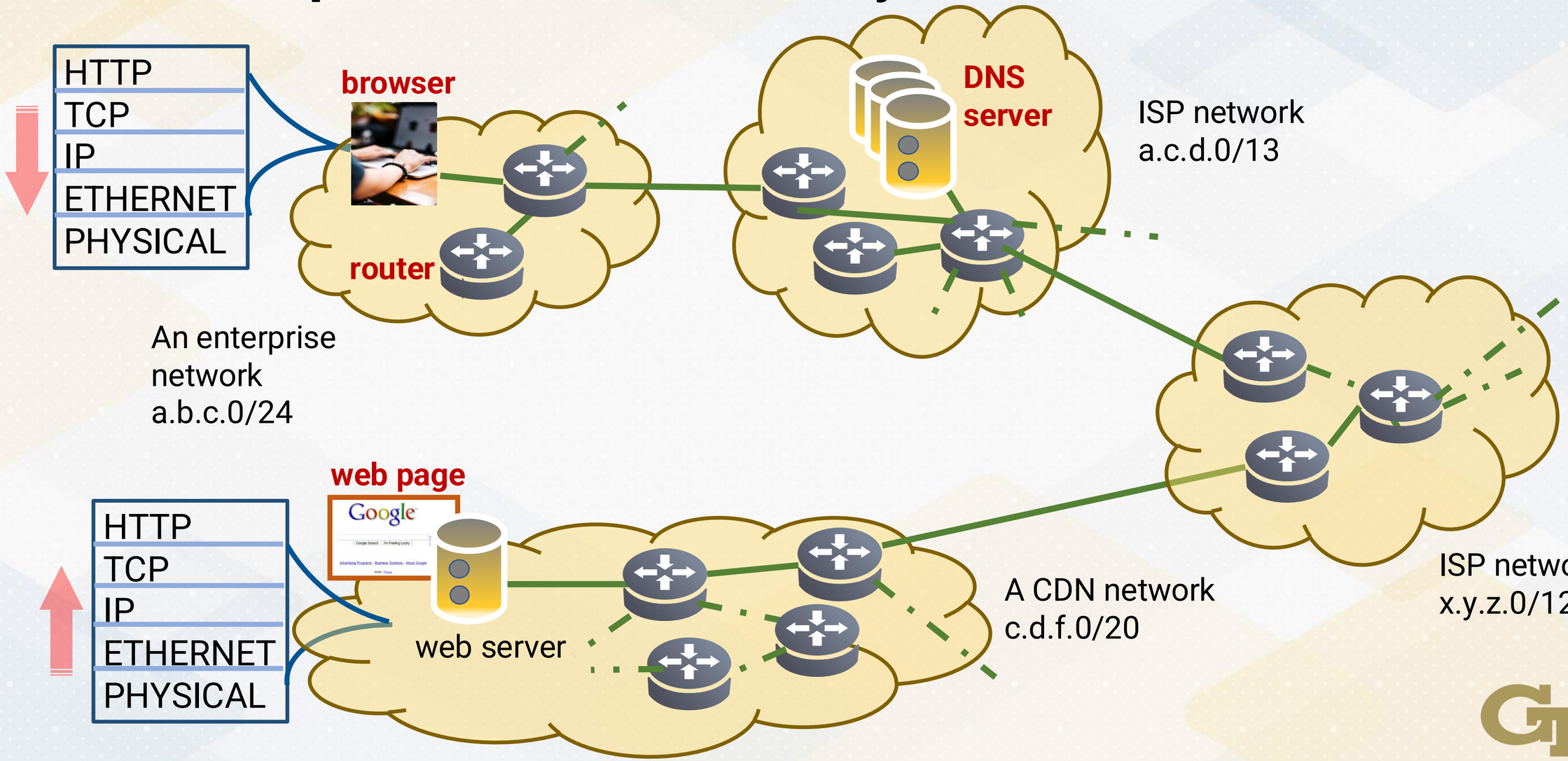
Georgia Tech

Reviewing the Internet's Protocol Stack



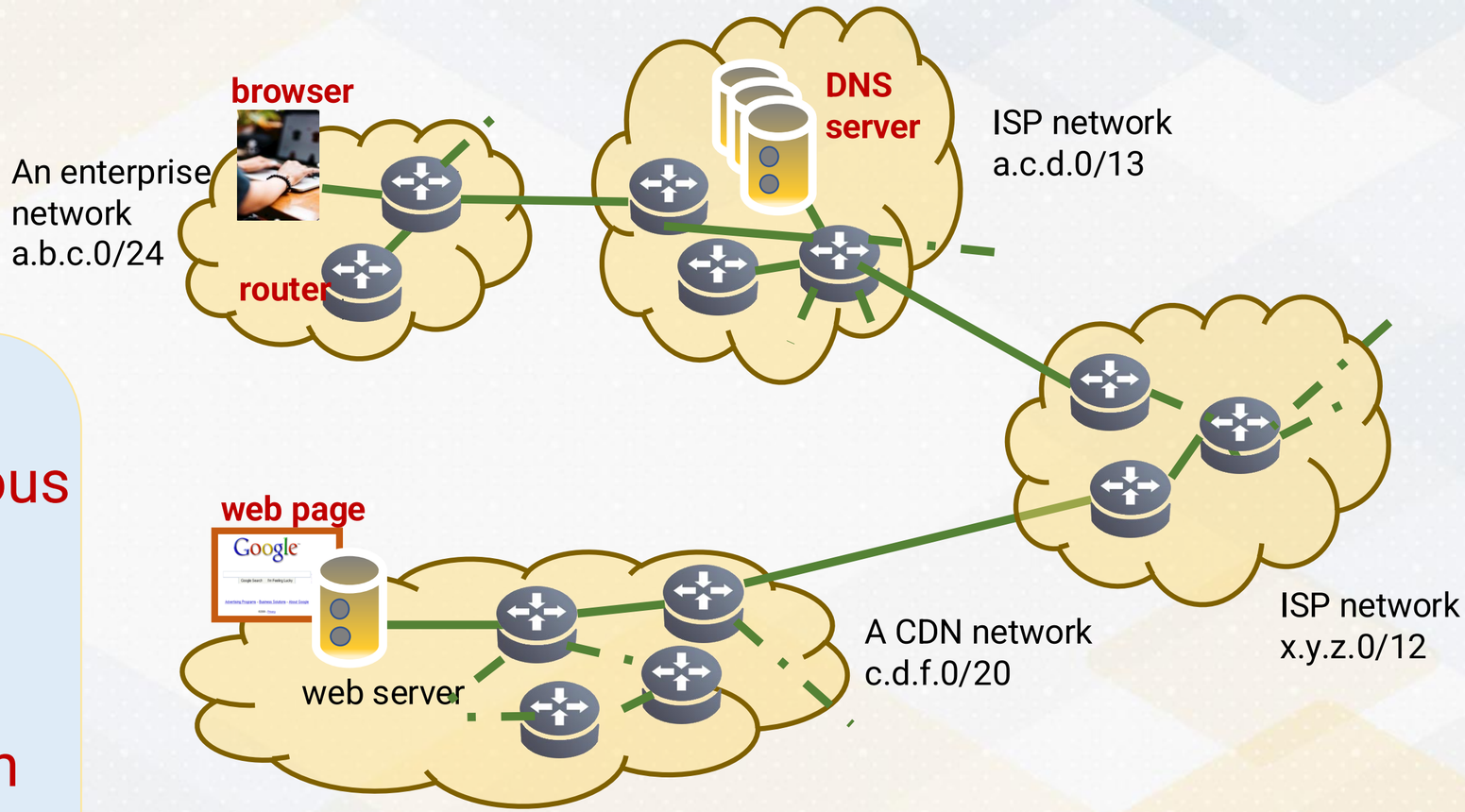
Georgia Tech

Reviewing the Internet's Protocol Stack
From the Transport to the Network Layer



Georgia Tech

Reviewing the Internet's Protocol Stack
Traffic Travels Through Multiple ASes

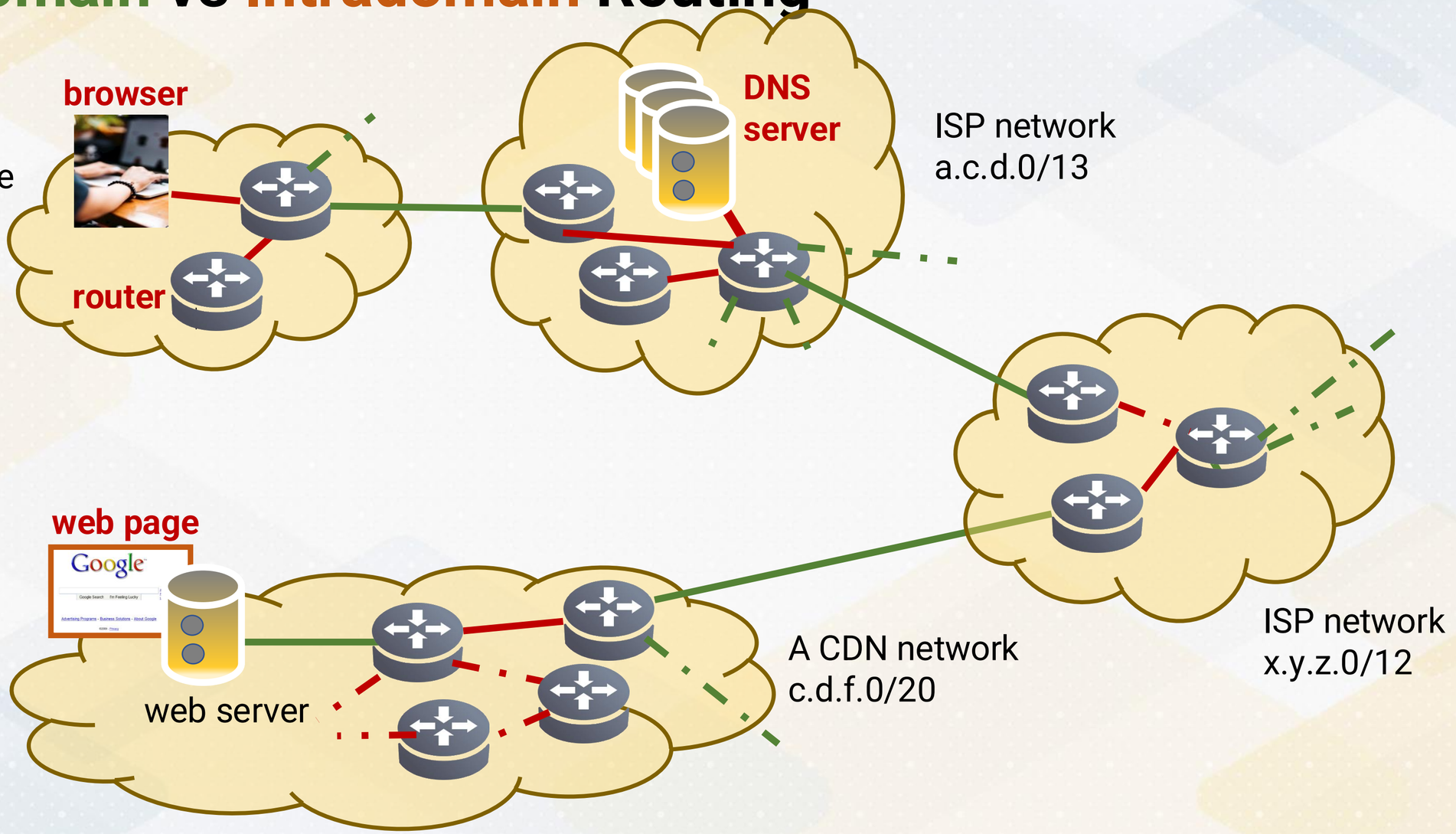


- End-to-end traffic crosses multiple Autonomous Systems (ASes)
- Each AS makes routing decisions **independently**
- Two routing tasks: within an AS vs between ASes

Georgia Tech

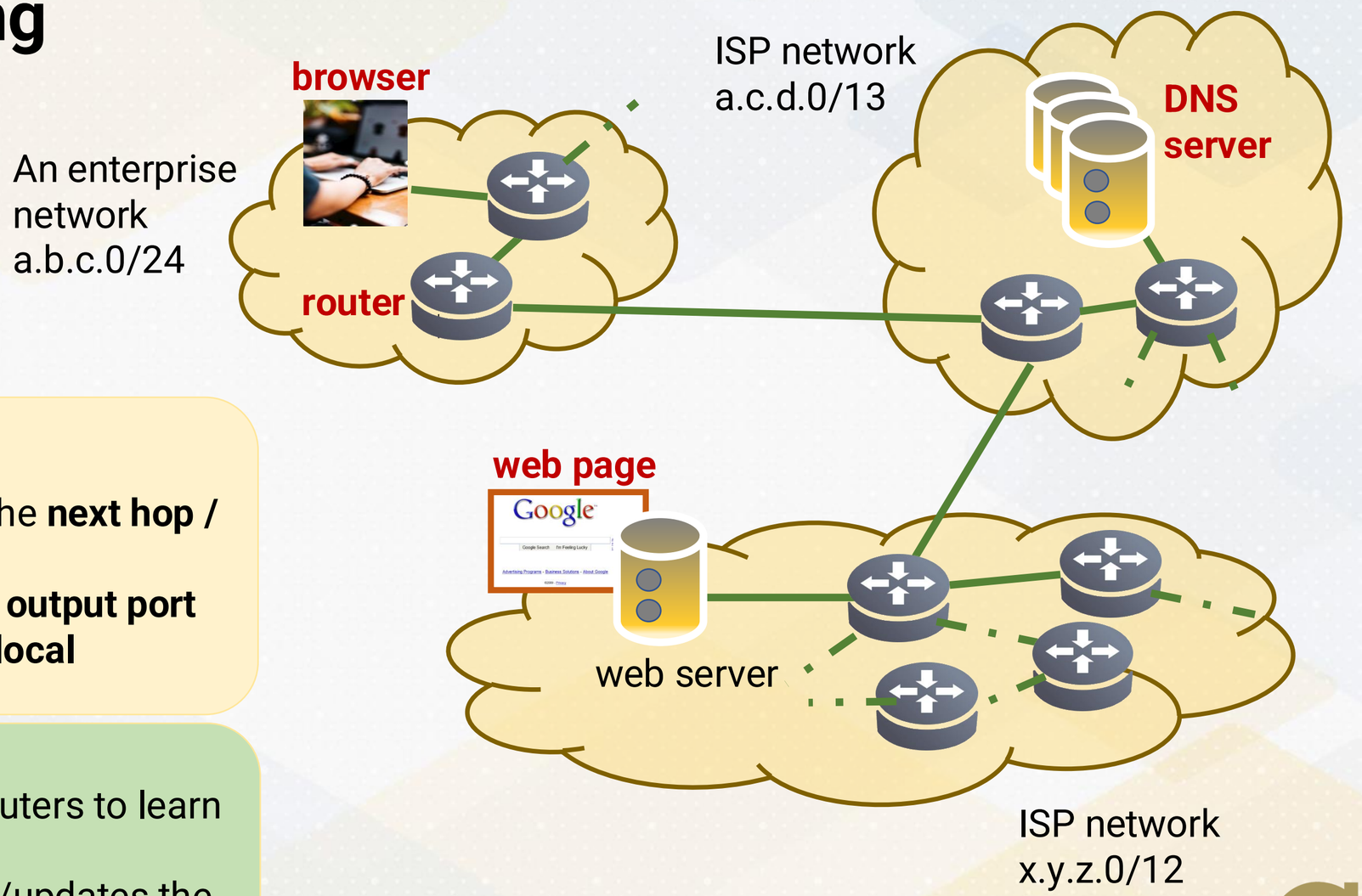
The Network Layer

Interdomain vs Intradomain Routing



Georgia Tech

The Router's Tasks
Routing & Forwarding



Forwarding (data plane)

- Uses the forwarding table to select the **next hop / outgoing interface**
- Moves the packet from **input port to output port**
- Operations are **per packet, fast and local**

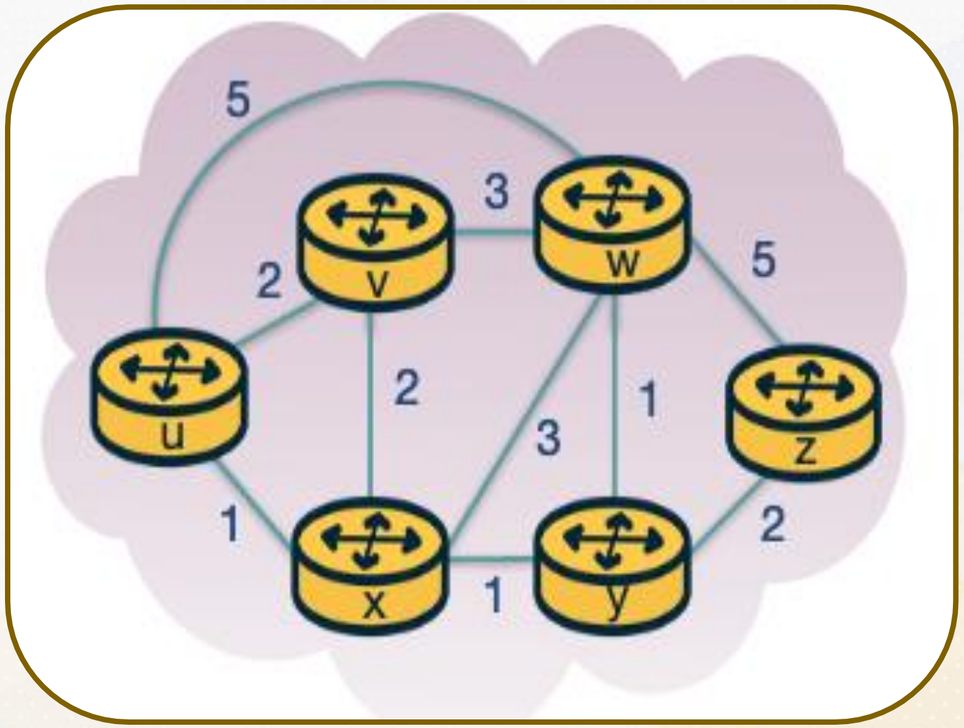
Routing (control plane)

- Exchanges information with other routers to learn about **paths/topology**
- Computes the **best paths** and builds/updates the forwarding table
- Adapts to **changes** (failures, cost updates)

Georgia Tech

Link Stating Routing (1/3)

- Routers share a **common view of the network topology** (who is connected to whom)
- Each link has a **cost metric** (delay, bandwidth, admin-defined cost)
- A **path** is the sequence of routers/links a packet follows from source to destination
- The **best path** is the path with the lowest total cost (based on that metric)
- The **forwarding table** stores the **next hop** for each destination—the first step on the best path



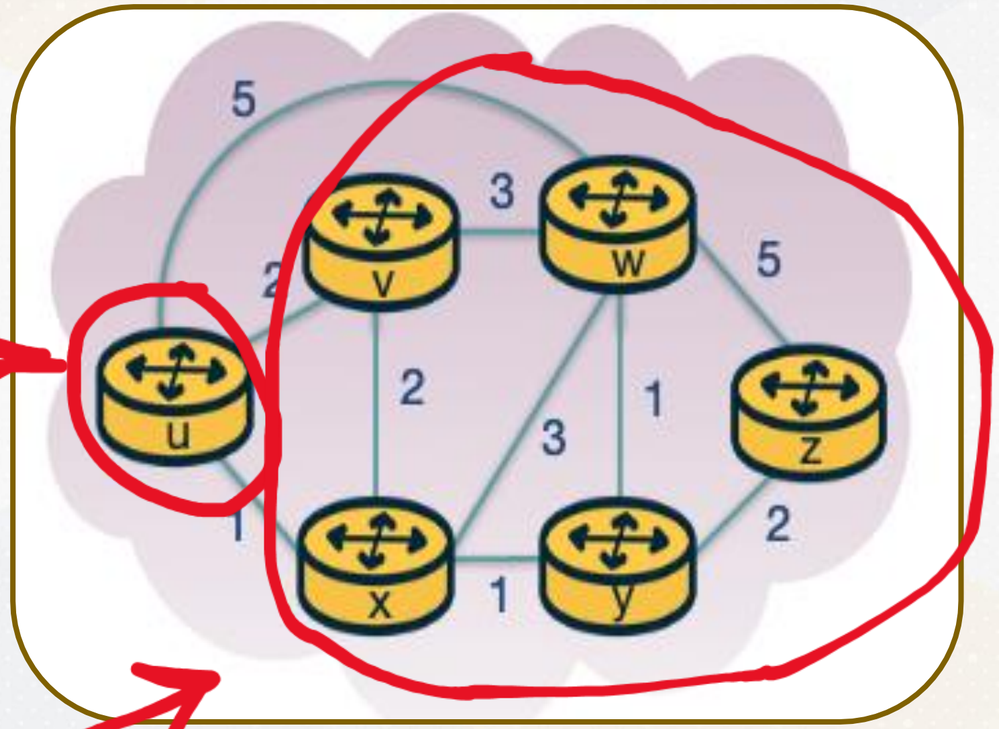
Goal: use that shared map to find best paths and fill the forwarding table.

Georgia Tech

Link Stating Routing (2/3)

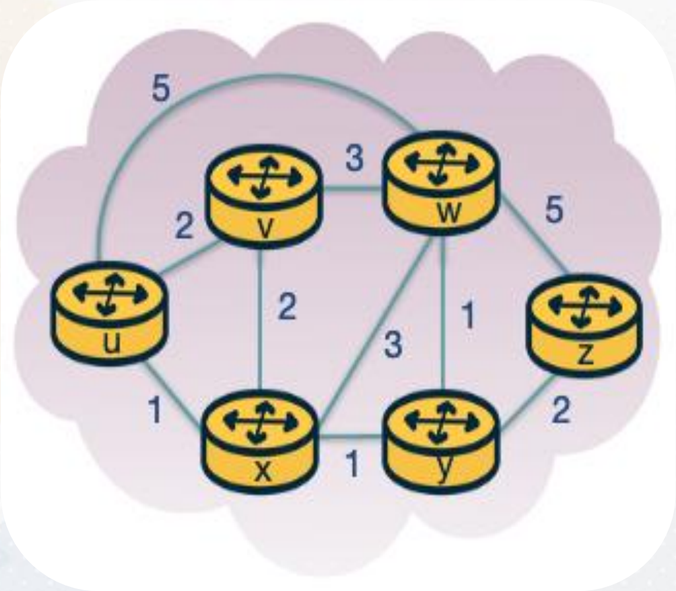
U: source node

V: every other node in the network

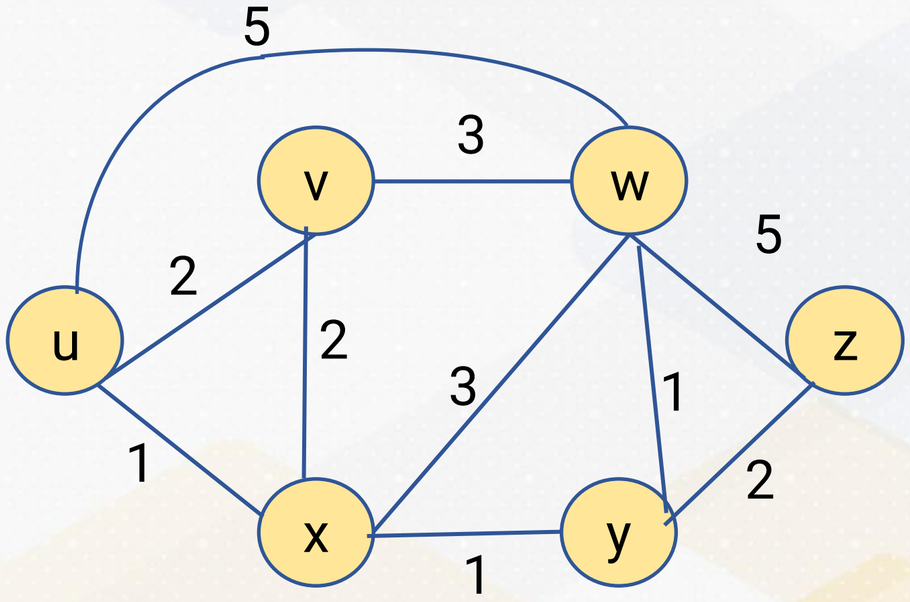


Georgia Tech

Link Stating Routing (3/3)



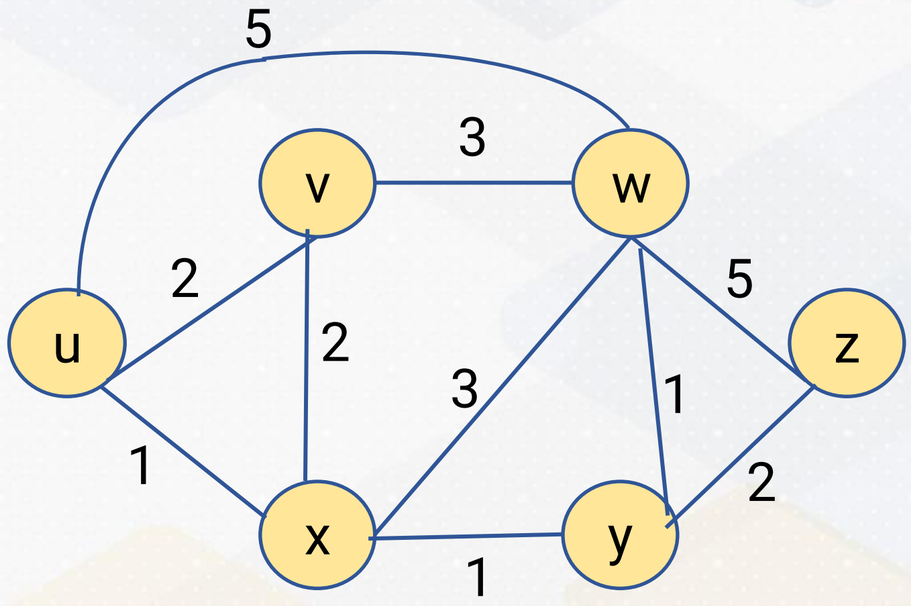
Graph representation



Georgia Tech

Link Stating Routing (1/3)
Dijkstra's Algorithm

- Keep track of **D(v)**: current best-known cost from source **u** to each node **v**
- Keep track of **p(v)**: predecessor on the best-known path (to reconstruct the route)
- Keep track of **N'**: set of nodes whose shortest paths from **u** are confirmed
- **Initialize:** N' = {u}; set D(neighbor) = link cost, and D(all others) = ∞ (with p(neighbor)=u)

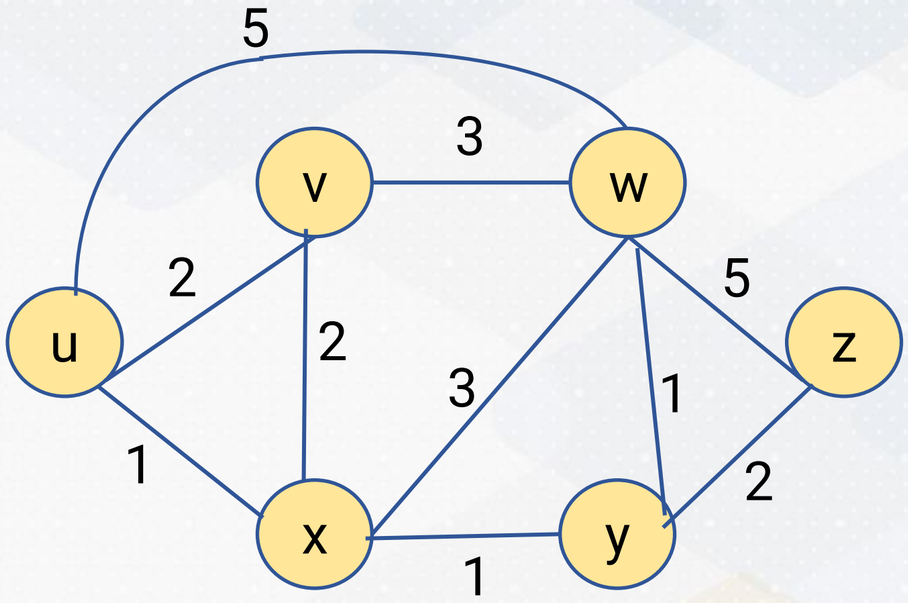


step	N’	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)

Georgia Tech

Link Stating Routing (2/3)
Dijkstra's Algorithm

- Keep track of **D(v)**: current best-known cost from source **u** to each node **v**
- Keep track of **p(v)**: predecessor on the best-known path (to reconstruct the route)
- Keep track of **N'**: set of nodes whose shortest paths from **u** are confirmed
- **Initialize:** N' = {u}; set D(neighbor) = link cost, and D(all others) = ∞ (with p(neighbor)=u)

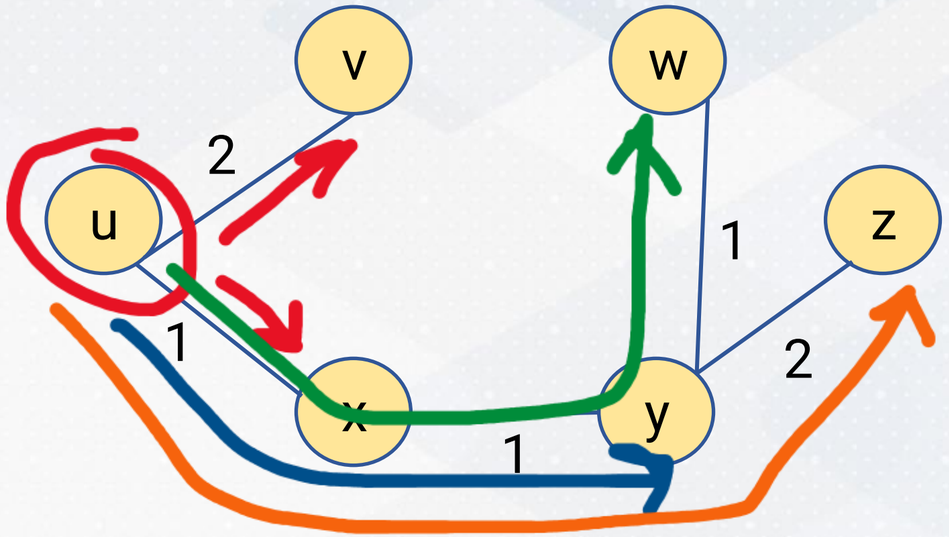


step	N’	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2, u	5, u	1, u	∞	∞

Georgia Tech

Link Stating Routing (3/3)
Dijkstra's Algorithm

- **When it ends**: we're done once **all nodes are in N'**, meaning the shortest path from the source to every destination is confirmed
- **Final result**: the algorithm produces a **shortest-path tree** rooted at the source; the router then builds the **forwarding table** by taking, for each destination, the **first hop** on that path
- **Complexity (why O(N²)):** Scan N−1, N−2, …, 1 nodes to find the next minimum ⇒ total ≈ N(N−1)/2.

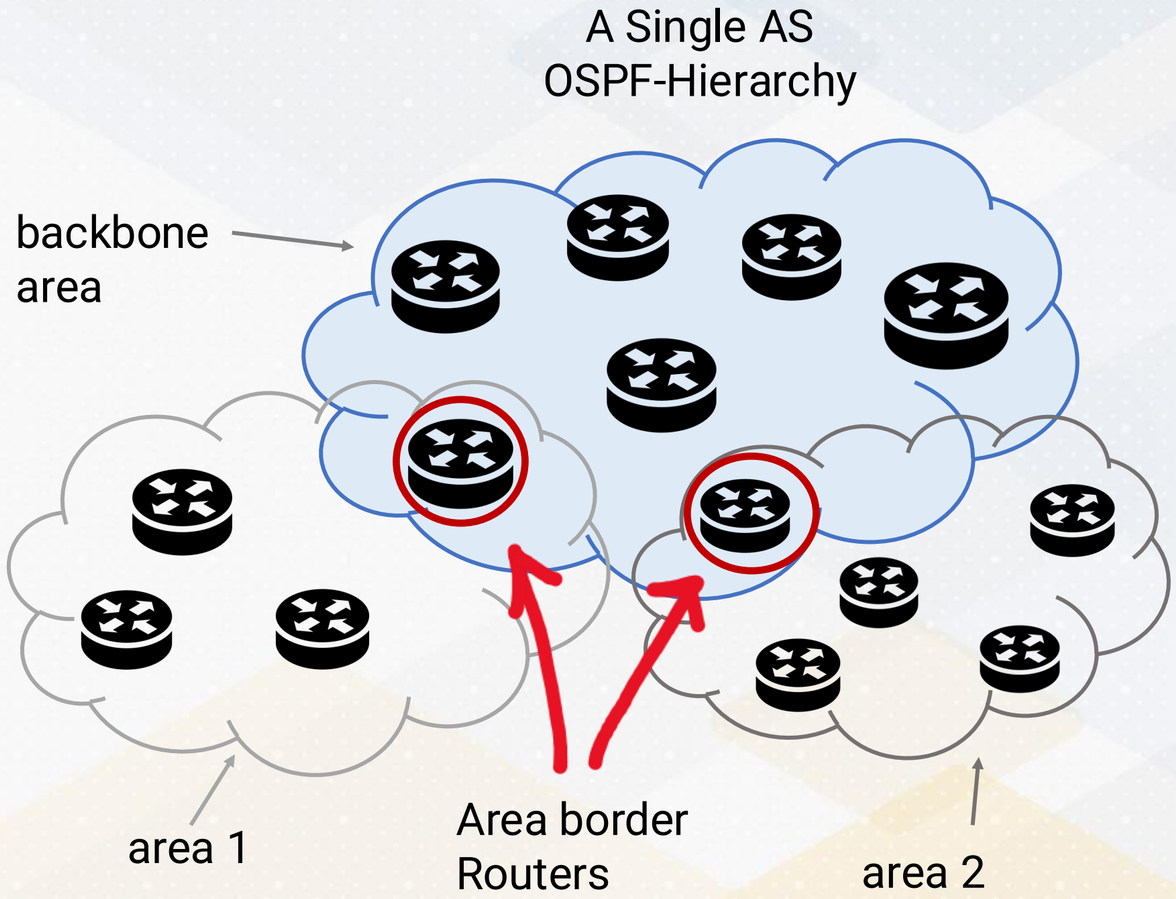


step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2,x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

Georgia Tech

Link Stating Routing
OSPF

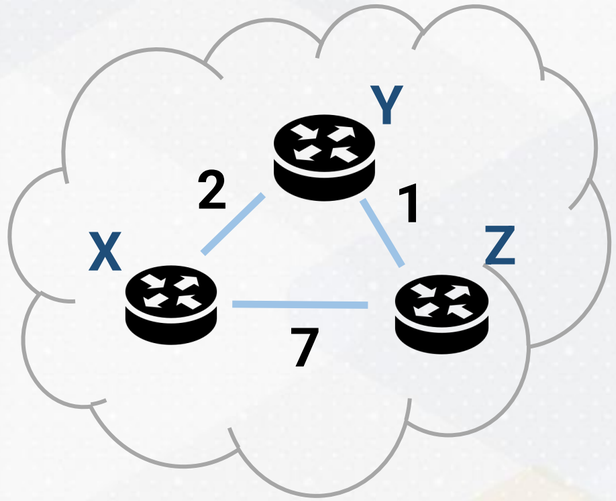
- OSPF is **an intradomain** (IGP, internal gateway routing protocol**) link-state protocol** used inside a single administrative domain (an AS)
- **Hierarchy for scale**: OSPF uses areas + a backbone, connected by Area Border Routers (ABRs)
- Routers flood **link-state advertisements (LSAs)** so routers in an area build the same topology database
- **Each router runs Dijkstra locally** and installs next hops into its forwarding table



Georgia Tech

Distance Vector Routing (1/3)

- **Goal**: compute least-cost routes **without** a global network map
- Each router keeps **a distance vector**: its current cost to every destination
- **Routers exchange vectors with neighbors only**; information "diffuses" hop-by-hop



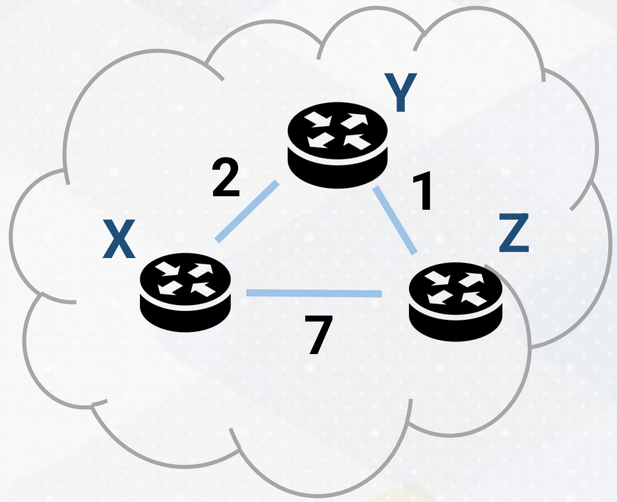
Bellman-Ford equation: $D_x(y) = \min_v\{c(x,v) + D_v(y)\}$

X's best cost to destination y is the minimum, over all neighbors v, of (cost to reach v) + (v's best cost to reach y)

Georgia Tech

Distance Vector Routing (2/3)

- Routers exchange **distance vectors with neighbors**
- Update rule: **cost via neighbor = link cost + neighbor's advertised cost**
- If a best **cost changes**, **update the table and re-advertise**
- Information spreads **hop-by-hop through the network**



Node x table

From \ Cost to	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

Node y table

From \ Cost to	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

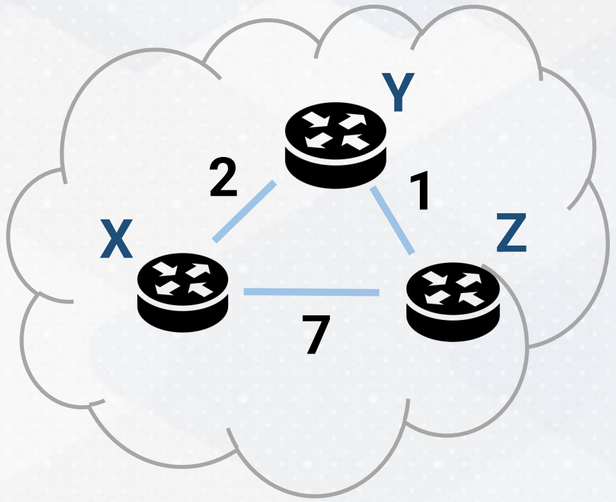
Node z table

From \ Cost to	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

Georgia Tech

Distance Vector Routing (3/3)

- **Converges** when no router has any new updates to send
- **Final state**: each router knows its **least-cost distance** to every destination
- Practical output: choose the **next hop** that achieves that best cost
- Example: x→z becomes **3 via y** (better than the direct cost 7)



Node x table

From \ Cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

Node y table

From \ Cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

Node z table

From \ Cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

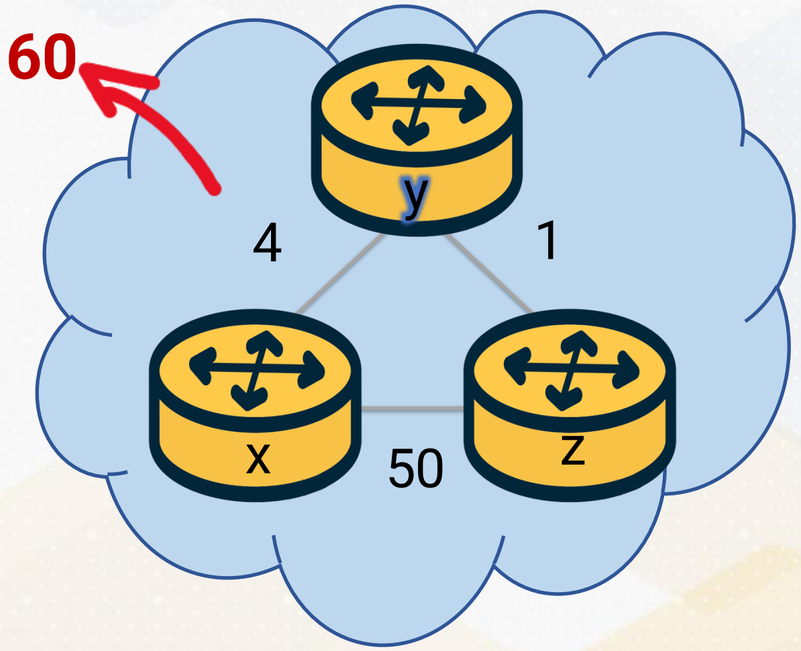
Georgia Tech

Distance Vector Routing
Count to Infinity

- **Trigger: a link cost increases** (or a link fails), “**bad news travels slowly**” in DV.
- Neighbors can form a temporary loop by trusting each other’s outdated distances

- After failure: Y routes to X via Z; Z routes to X via Y (mutual misconception)
- Routers **increment the advertised cost step-by-step** (6, 7, 8, …) toward “infinity”

- **Convergence can take many rounds** until a router switches to a **real alternative** (or hits a max metric)

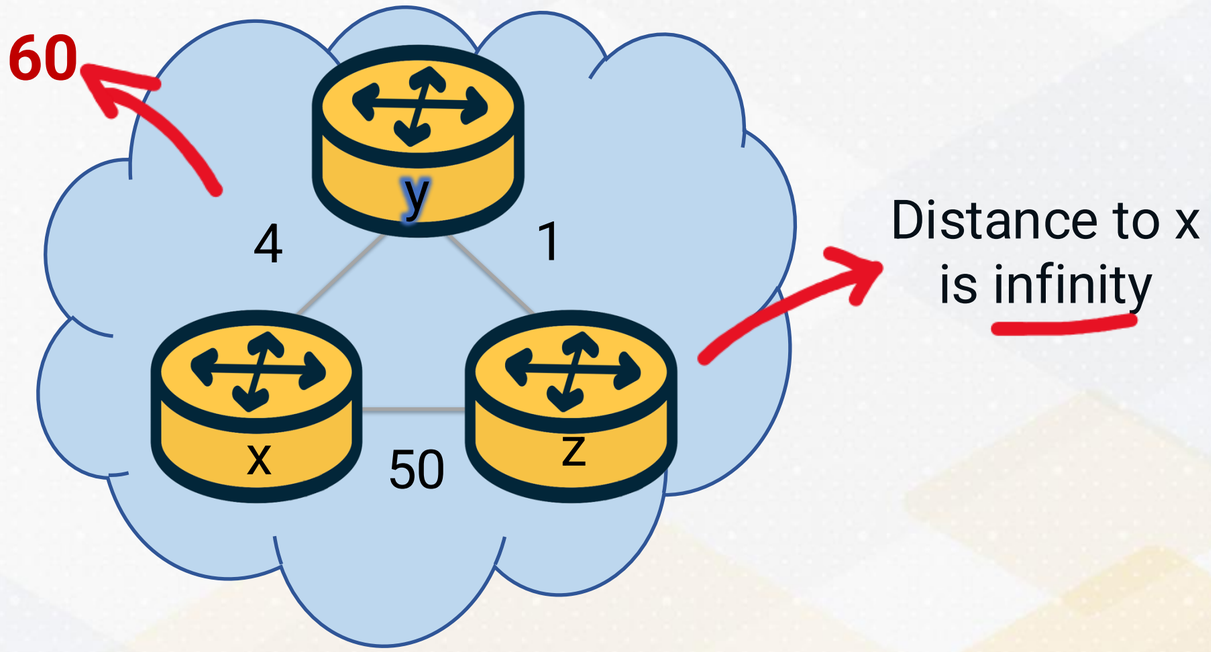


Georgia Tech

Distance Vector Routing
Count to Infinity → Poisson Reverse

- **If I route to destination x via neighbor y, I tell y: "my distance to x is ∞"**

- **Prevents two-node loops** (e.g. "you route through me, I route through you")
- **Speeds up convergence** after failures/cost increases in common cases
- **Limitation**: doesn't fully fix multi-node count-to-infinity scenarios



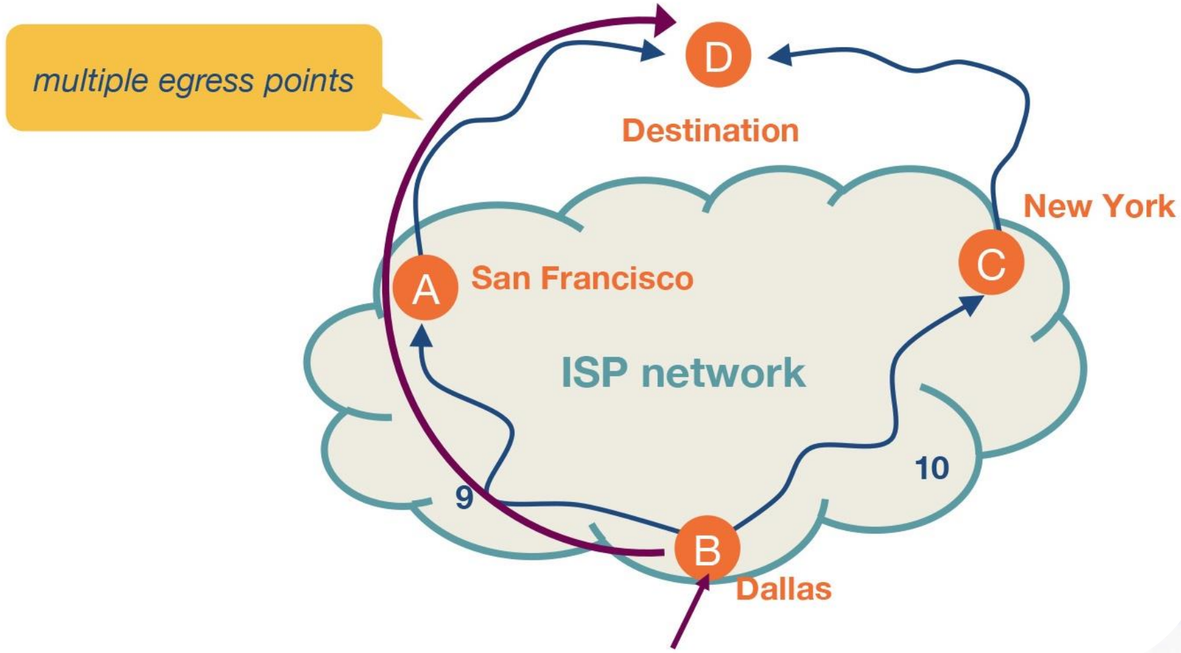
Georgia Tech

Hot Potato Routing

When Intra Can Affect Inter-domain Routing

From intra to inter: once a destination is *outside* the domain, the router must choose an **egress point.** That is an intradomain choice that directly shapes the interdomain.

Hot Potato Routing Rule: pick the closest exit based on **intradomain (IGP) cost** to reach that egress (“closest-exit routing”)
Motivation: hand traffic off quickly and reduce internal bandwidth or operational load (“get it out of my network”)
Impact: small IGP changes can shift the chosen exit and cause **traffic shifts.**



Teixeira, R., Shaikh, A., Griffin, T., & Rexford, J. (2004, June). Dynamics of hot-potato routing in IP networks. In *Proceedings of the joint international conference on Measurement and modeling of computer systems* (pp. 307-319).

Georgia Tech

Summary

- TCP/UDP manage end-to-end delivery; IP carries packets router-to-router
- Inside a router: forwarding = per-packet lookup and send; routing = compute/maintain the lookup table
- Across the Internet: packets traverse multiple organizations (enterprise to ISPs to CDNs), so routing spans within and across administrative domains
- Intradomain routing approaches:
  - Link-state: flood link info, run Dijkstra, install next hops
  - Distance-vector: trade vectors with neighbors, apply Bellman-Ford, converge over iterations
- In practice: convergence issues (count-to-infinity, poison reverse), and policy-driven behavior -> closest-exit (hot potato) routing

Georgia Tech